Prolog Programming

Chapter 3

Lists, Operators, Arithmetic

28.07.2020

Chapter 3
Lists, Operators, Arithmetic

- Representation of lists
- Some operations on lists
- Operator notation
- Arithmetic

3.1 Representation of lists

- The list is a simple data structure widely used in non-numeric programming
- A list is a sequence of any number of items such as ann, tennis, tom, skiing
- In Prolog,

  [ann, tennis, tom, skiing]

List as Object in Prolog

- Two cases
  - Empty list
  - Non-empty list
- List is written as a Prolog atom, [ ]
- A non-empty list contains two things
  - head, the first item of the list
  - tail, the remaining part of the list

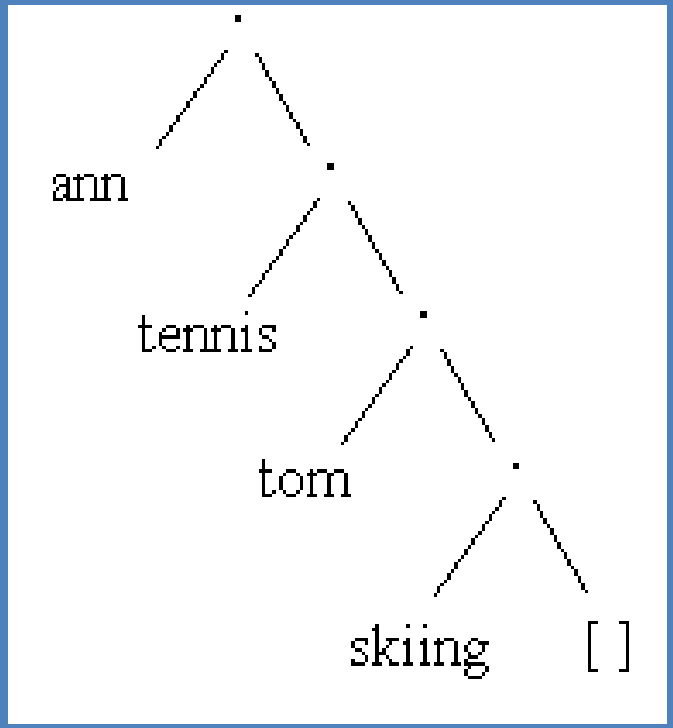
List looks like a tree

- List can be written as
- .(Head, Tail)

For example,

- [ann, tennis, tom, skiing] can be written as
- .(ann, .(tennis, .(tom, .(skiing, [ ]))))

Tree representation of the list [ann,tennis,tom,skiing]



Examples

?- List1 = [a,b,c] ,
List2 = . (a , . (b , . (c , [ ])))).

List1 = [a,b,c]
List2 = [a,b,c]

?- Hobbies1 = . (tennis, . (music, [ ])),
Hobbies2 = [skiing, food],
L = [ann, Hobbies1, tom, Hobbies2]

Hobbies1 = [tennis, music]
Hobbies2 = [skiing, food]
L = [ann, [tennis, music], tom, [skiing, food]]

More Examples

If L = [a, b, c]

- Tail = [b, c] and L = . (a, Tail)

Another notation,

- L = [a | Tail]
- [a, b, c] = [a | [b, c]] = [a, b, [c]] = [a, b, c, | [ ]]

In summary, lists can be written as

- [Item1, Item2, …]
- [Head | Tail]
- [Item1, Item2, … | Others]

Chapter 3
Lists, Operators, Arithmetic

- Representation of lists
- Some operations on lists
- Operator notation
- Arithmetic

3.2 Some Operations

- Checking membership in a list
- Concatenating two lists
- Adding an item to a list and deleting an item from a list

Membership in a list

- Define the membership as

  member (X , L)

  where X is an object and L is a list

- For example

  member (b, [a, b, c]) true

  member (b, [a, [b, c]]) not true

  member ([b, c], [a, [b, c]]) true

Membership in a list continues ---

- X is a member of L if either:

  (1) X is the head of L, or

  (2) X is a member of the tail of L

- In Prolog,

```
member ( X, [ X | Tail ] ).
member ( X, [ Head | Tail ] ) :-
                member ( X, Tail ).
```

Membership in a list continues ---

- Questions to Prolog,

  ?- member (b, [a, b, c]) .
  - Yes

  ?- member (X, [a, b, c]) .
  - X=a
  - X=b
  - X=c

3.2 Some Operations

- Checking membership in a list
- Concatenating two lists
- Adding an item to a list and deleting an item from a list

Concatenating two lists

- Define the relation as follow:

  conc(L1,L2,L3)

  where L1 and L2 are two lists and L3 is their concatenation

- In Prolog,

```
conc([ ], L1, L1).
conc([X|L1],L2,[X|L3] :-
                conc(L1,L2,L3).
```

Concatenating two lists continues ---

- Question to Prolog

  ?- conc ([a, b, c] , [d, e, f], L) .

  L = [a, b, c, d, e, f]

- Also concatenation relation can be used in the inverse direction for decomposition a given list into two lists

Decomposition a given list into two lists using concatenation

?- conc(L1,L2,[a,b,c]).

- L1=[ ]
- L2=[a,b,c]

- L1=[a]
- L2=[b,c]

- L1=[a,b]
- L2=[c]

- L1=[a,b,c]
- L2=[ ]

Look for a certain pattern in a list using concatenation

```
?- conc(Before,[may|After],
        [jan,feb,mar,apr,may,jun,jul,aug,sep,oct,nov,dec]).
```

- Before=["jan","feb","mar","apr"]
- After=["jun","jul","aug","sep","oct","nov","dec"

Delete from a list using concatenation

```
?- L1=[a,b,z,z,c,z,z,z,d,e],
   conc(L2,[z,z,z|_],L1).
```

- Delete everything that follows three successive occurrences of z in list L1
- L1=["a","b","z","z","c","z","z","z","d","e"],
  L2=["a","b","z","z","c"]

Check membership using concatenation

```
member1 ( X , L ) :-
        conc ( _ , [ X | _ ] , L ) .

?- member1(X,[a,b,c])
```

- X=a
- X=b
- X=c

Adding an item

- In Prolog,

  add (X, L, [X | L]) .

  where X is a new item and it is added to the list L

- Question

  ?- add (a, [b, c], L) .

- L=["a","b","c"]

Deleting an item

- Deleting an item X from a list L

  del (X, L, L1).

  where L1 is a new list after X is removed from the list L

- There are two cases as follows:

- (1) If X is the head of the list then the result after the deletion is the tail of the list

- (2) If X is in the tail then it is deleted from there

Deleting an item continue ---

- In Prolog,

```
del ( X, [ X | Tail ], Tail ) .                 % case 1
del ( X, [ Y | Tail ], [ Y | Tail1 ] ) : -      % case 2
                del ( X, Tail,Tail1 ) .
```

- Question

```
?- del ( a, [a, b, a, a ], L ).
```

  - L=["b","a","a"]
  - L=["a","b","a"]
  - L=["a","b","a"]

Add an item to a list by using delete

- “del” can be used in the inverse direction, to add an item to a list by inserting the new item anywhere in the list
- What is L, such that after deleting 5 from L, obtain [2,3,4]?
- Question

  ?- del (5, L, [2, 3, 4]) .
  - L=[5,1,2,3]
  - L=[1,5,2,3]
  - L=[1,2,5,3]
  - L=[1,2,3,5]

Inserting an item to a list and checking membership using delete

- We define “insert” relation as follows:

```
insert ( X, List, BiggerList ) :-
                    del ( X, BiggerList, List ) .
```

- We can also define membership relation using del

```
member2 ( X, List ) :-
                    del ( X, List, _ ) .
```